

- Description

This module manages a smart subway station passenger flow monitoring system, ensuring that:

1. **The number of passengers entering and exiting the station is continuously monitored.**
2. **If the station reaches maximum capacity, turnstiles restrict new entries until the congestion level decreases.**
3. **If the station is underutilized, more turnstiles open to improve passenger flow efficiency.**
4. **A log of all passenger counts, congestion alerts, and turnstile adjustments is maintained for real-time management.**

This system helps prevent overcrowding, optimize station efficiency, and enhance passenger safety.

- Specification

module SmartSubwayStationMonitoringEx

var currentPassengers, maxCapacity, openTurnstiles: integer

var log: sequence of string

process enterStation(numPassengers: integer)

functional scenario form:

currentPassengers + numPassengers <= maxCapacity && currentPassengers = currentPassengers + numPassengers, adjustTurnstiles(), log = log $\oplus$ ("Entry: " + numPassengers + " passengers | New Count: " + currentPassengers + " | Open Turnstiles: " + openTurnstiles), Passengers entered, turnstile count adjusted.

||

currentPassengers + numPassengers > maxCapacity && log = log $\oplus$ ("Overcrowding Alert: Entry Restricted."), Overcrowding alert triggered, entry restricted.

end process

process exitStation(numPassengers: integer)

functional scenario form:

currentPassengers = max(0, currentPassengers - numPassengers), adjustTurnstiles(), log = log $\oplus$ ("Exit: " + numPassengers + " passengers | New Count: " + currentPassengers + " | Open Turnstiles: " + openTurnstiles) Passengers exited, turnstile count adjusted.

end process

end module

- Java Code

```
package code;
import java.util.*;
import java.util.concurrent.ThreadLocalRandom;
import java.util.function.Supplier;
```

```

/**
 * SubwayPassengerFlowSystemEx
 *
 * SmartWell-structured demo:
 * - Core domain model (SubwayStation, Turnstile, TicketGatePolicy)
 * - Sensors & Probes (FlowSensor, CrowdSensor)
 * - Controller (FlowController)
 * - Alerts & Logging (AlertType, Alert, StationLogger)
 * - Simulation Engine (Scenario, Runner)
 * - Replay & Deterministic RNG support
 * - VT / FT / Z3 / UVT test suites (toggle by CLI arg or const)
 * - Simple CLI flags: MODE=VT|FT|Z3|UVT, SEED=<long>, LOG=on|off
 *
 * Notes:
 * - The "enterStation/exitStation" logic remains faithful to your bas
eline.
 * - We keep overcrowding restriction on entry; exits always allowed (
floor at 0).
 * - Turnstile policy adapts with crowd density: >80% => 3, <30% => 8,
otherwise 5.
 * - Added soft policies (peak-
hour bias, emergency compression) for realism.
 */
public class c123_SubwayPassengerFlowSystemEx {
    /* =====
    * ===== Domain Model =====
    * ===== */
    /** Encodes alert categories for station operations. */
    enum AlertType {
        OVERCROWDING,
        UNUSUAL_NEGATIVE_ENTRY,
        NEGATIVE_EXIT_REQUEST,
        TURNSTILE_POLICY_SWITCH,
        SENSOR_FAULT,
        INFO
    }
    /** Alert object with type, message, timestamp. */
    static final class Alert {
        final AlertType type;
        final String message;
        final long ts;
        Alert(AlertType type, String message) {
            this.type = type;
            this.message = message;

```

```

        this.ts = System.currentTimeMillis();
    }
    @Override
    public String toString() {
        return "[" + type + "]" + message + " @ " + ts;
    }
}

/** Centralized logger used by station + controller + engine. */
static final class StationLogger {
    private final List<String> lines = new ArrayList<>();
    private final List<Alert> alerts = new ArrayList<>();
    private boolean console = true;
    void setConsole(boolean on) { this.console = on; }
    void log(String s) {
        lines.add(s);
        if (console) System.out.println(s);
    }
    void alert(AlertType t, String msg) {
        Alert a = new Alert(t, msg);
        alerts.add(a);
        if (console) System.out.println("ALERT: " + a);
    }
    List<String> lines() { return Collections.unmodifiableList(line
s); }
    List<Alert> alerts() { return Collections.unmodifiableList(aler
ts); }
    void printSummary() {
        log("\n=== Logger Summary ===");
        log("Total lines: " + lines.size());
        log("Total alerts: " + alerts.size());
        Map<AlertType, Long> cnt = new EnumMap<>(AlertType.class);
        for (Alert a : alerts) {
            cnt.put(a.type, cnt.getDefault(a.type, 0L) + 1);
        }
        for (AlertType t : AlertType.values()) {
            long v = cnt.getDefault(t, 0L);
            log("  - " + t + ": " + v);
        }
    }
}

/** Simple turnstile entity (for extensibility). */
static final class Turnstile {
    final int id;
    boolean open;

```

```

    Turnstile(int id) {
        this.id = id;
        this.open = true;
    }
}

/** Ticket gate policy: maps passenger density to number of open tu
rnstiles. */
static final class TicketGatePolicy {
    final double highDensity; // e.g., 0.80
    final double lowDensity;  // e.g., 0.30
    final int highModeOpen;   // e.g., 3
    final int lowModeOpen;    // e.g., 8
    final int normalOpen;     // e.g., 5
    TicketGatePolicy(double highDensity, double lowDensity, int hig
hModeOpen, int lowModeOpen, int normalOpen) {
        this.highDensity = highDensity;
        this.lowDensity = lowDensity;
        this.highModeOpen = highModeOpen;
        this.lowModeOpen = lowModeOpen;
        this.normalOpen = normalOpen;
    }
    int decide(int maxCapacity, int currentPassengers) {
        if (maxCapacity <= 0) return normalOpen;
        double ratio = (double) currentPassengers / (double) maxCap
acity;
        if (ratio > highDensity) return highModeOpen;
        if (ratio < lowDensity) return lowModeOpen;
        return normalOpen;
    }
}

/** Statistics collector for one simulation run. */
static final class StationStats {
    long totalEntries;
    long totalExits;
    long deniedEntries;
    long negativeEntryCalls;
    long negativeExitCalls;
    long policySwitches;
    int maxObservedPassengers;
    int minObservedPassengers;
    StationStats(int initial) {
        this.maxObservedPassengers = initial;
        this.minObservedPassengers = initial;
    }
}

```

```

        void onCount(int current) {
            maxObservedPassengers = Math.max(maxObservedPassengers, current);
            minObservedPassengers = Math.min(minObservedPassengers, current);
        }
        void print(StationLogger log) {
            log.log("\n=== Station Stats ===");
            log.log("Total Entries Accepted: " + totalEntries);
            log.log("Total Exits: " + totalExits);
            log.log("Denied Entries (overcrowding): " + deniedEntries);
            log.log("Negative Entry Calls: " + negativeEntryCalls);
            log.log("Negative Exit Calls: " + negativeExitCalls);
            log.log("Policy Switches: " + policySwitches);
            log.log("Max Observed Passengers: " + maxObservedPassengers);
        );
        log.log("Min Observed Passengers: " + minObservedPassengers);
    );
    }
}

/** Core station model (preserves your original logic and API). */
static final class SubwayStation {
    private final int maxCapacity;
    private int currentPassengers;
    private int openTurnstiles;
    private final List<String> log;
    private final StationLogger sharedLogger;
    private final TicketGatePolicy policy;
    private final StationStats stats;
    SubwayStation(int maxCapacity, int initialPassengers, TicketGatePolicy policy, StationLogger sharedLogger) {
        this.maxCapacity = maxCapacity;
        this.currentPassengers = initialPassengers;
        this.openTurnstiles = 5; // default
        this.log = new ArrayList<>();
        this.sharedLogger = sharedLogger;
        this.policy = policy;
        this.stats = new StationStats(initialPassengers);
    }
    int getCurrentPassengers() { return currentPassengers; }
    int getMaxCapacity() { return maxCapacity; }
    int getOpenTurnstiles() { return openTurnstiles; }
    StationStats stats() { return stats; }
}
/**

```

```

        * Simulates passenger entry. If overcrowding would happen, deny entry.
        * Negative inputs are treated as abnormal and counted, but applied as-is (as in your test lists).
        */
        public void enterStation(int numPassengers) {
            if (numPassengers < 0) {
                sharedLogger.alert(AlertType.UNUSUAL_NEGATIVE_ENTRY, "Negative entry request: " + numPassengers);
                stats.negativeEntryCalls++;
            }
            if (currentPassengers + numPassengers > maxCapacity) {
                sharedLogger.alert(AlertType.OVERCROWDING, "Entry denied. current=" + currentPassengers
                    + ", try=" + numPassengers + ", max=" + maxCapacity);
                log.add("Overcrowding Alert: Entry Restricted.");
                stats.deniedEntries++;
                return;
            }
            currentPassengers += numPassengers;
            adjustTurnstiles();
            String line = String.format("Passengers Entered: %d | Current Count: %d | Open Turnstiles: %d",
                numPassengers, currentPassengers, openTurnstiles);
            sharedLogger.log(line);
            log.add("Entry: " + numPassengers + " | New Count: " + currentPassengers + " | Open Turnstiles: " + openTurnstiles);
            stats.totalEntries += Math.max(0, numPassengers);
            stats.onCount(currentPassengers);
        }
        /**
        * Simulates passenger exit; the count never goes below zero (as in your baseline).
        * Negative exits are unusual; we log & count them but apply the Math.max floor.
        */
        public void exitStation(int numPassengers) {
            if (numPassengers < 0) {
                sharedLogger.alert(AlertType.NEGATIVE_EXIT_REQUEST, "Negative exit request: " + numPassengers);
                stats.negativeExitCalls++;
            }
            int before = currentPassengers;

```

```

        currentPassengers = Math.max(0, currentPassengers - numPassengers);
        adjustTurnstiles();
        String line = String.format("Passengers Exited: %d | Current Count: %d | Open Turnstiles: %d",
            numPassengers, currentPassengers, openTurnstiles);
        sharedLogger.log(line);
        log.add("Exit: " + numPassengers + " | New Count: " + currentPassengers + " | Open Turnstiles: " + openTurnstiles);
        int actualExit = Math.max(0, before - currentPassengers);
        stats.totalExits += actualExit;
        stats.onCount(currentPassengers);
    }
    /** Applies the policy; logs policy change events. */
    private void adjustTurnstiles() {
        int desired = policy.decide(maxCapacity, currentPassengers);
;
        if (desired != openTurnstiles) {
            openTurnstiles = desired;
            stats.policySwitches++;
            sharedLogger.alert(AlertType.TURNSTILE_POLICY_SWITCH, "Policy set openTurnstiles=" + desired
                + " at " + currentPassengers + "/" + maxCapacity);
        }
    }
    /** Dumps embedded log collected by this station instance. */
    public void printLog() {
        sharedLogger.log("\nSubway Station Log:");
        for (String logEntry : log) {
            sharedLogger.log(logEntry);
        }
    }
}
/* =====
 * ===== Sensors/Probes =====
 * ===== */
/** Abstract sensor interface for extensibility. */
interface Sensor<T> {
    T read();
}
/** Sensor simulating passenger inflow/outflow suggestions (e.g., camera count). */
static final class FlowSensor implements Sensor<Integer> {

```

```

        private final Random rnd;
        private final int mean;
        private final int spread;
        private final boolean canNeg;
        FlowSensor(long seed, int mean, int spread, boolean canNeg) {
            this.rnd = new Random(seed);
            this.mean = mean;
            this.spread = Math.max(0, spread);
            this.canNeg = canNeg;
        }
        @Override
        public Integer read() {
            int delta = mean + (int) Math.round((rnd.nextGaussian() * s
pread));
            if (!canNeg) delta = Math.abs(delta);
            return delta;
        }
    }
    /** Sensor for crowd density (ratio). */
    static final class CrowdSensor implements Sensor<Double> {
        private final Supplier<Integer> current;
        private final Supplier<Integer> capacity;
        CrowdSensor(Supplier<Integer> current, Supplier<Integer> capaci
ty) {
            this.current = current;
            this.capacity = capacity;
        }
        @Override
        public Double read() {
            int cap = Math.max(1, capacity.get());
            return Math.min(1.0, Math.max(0.0, current.get() / (double)
cap));
        }
    }
    /** =====
    * ===== Controller =====
    * ===== */
    /** Controller adjusts entry/exit application strategies during sim
ulation. */
    static final class FlowController {
        private final SubwayStation station;
        private final CrowdSensor crowd;
        private final StationLogger log;
        // Optional knobs for realism:

```

```

        private boolean peakHourBias = true;
        private boolean emergencyCompression = true;
        FlowController(SubwayStation station, CrowdSensor crowd, StationLogger log) {
            this.station = station;
            this.crowd = crowd;
            this.log = log;
        }
        void setPeakHourBias(boolean on) { this.peakHourBias = on; }
        void setEmergencyCompression(boolean on) { this.emergencyCompression = on; }
        /**
         * Intercept incoming request and optionally reshape/clip based on policies.
         * For example:
         * - During peak hours, encourage exit preference.
         * - Under emergency compression, clip unusually large negative values.
         */
        int preprocessEntryRequest(int req, int minuteOfDay) {
            if (peakHourBias && isPeakHour(minuteOfDay) && req > 0) {
                // Slightly dampen huge entry at peak to simulate gate metering
                int clipped = (int) Math.round(req * 0.85);
                if (clipped != req) {
                    log.log("[Controller] Peak-hour entry metering: " + req + " -> " + clipped);
                }
                return clipped;
            }
            if (emergencyCompression && req < -1000) {
                int clipped = -1000;
                log.log("[Controller] Emergency compression on negative entry: " + req + " -> " + clipped);
                return clipped;
            }
            return req;
        }
        int preprocessExitRequest(int req, int minuteOfDay) {
            if (peakHourBias && isPeakHour(minuteOfDay) && req > 0) {
                // Encourage more exit throughput at peak
                int boosted = (int) Math.round(req * 1.10);
                if (boosted != req) {

```

```

        log.log("[Controller] Peak-
hour exit boost: " + req + " -> " + boosted);
    }
    return boosted;
}
if (emergencyCompression && req < -1000) {
    int clipped = -1000;
    log.log("[Controller] Emergency compression on negative
exit: " + req + " -> " + clipped);
    return clipped;
}
return req;
}
private boolean isPeakHour(int minuteOfDay) {
    // 7:30-9:30 and 17:00-19:30 as example peak windows
    int m = minuteOfDay % (24 * 60);
    return (m >= (7 * 60 + 30) && m <= (9 * 60 + 30)) ||
        (m >= (17 * 60) && m <= (19 * 60 + 30));
}
}
/* =====
 * ===== Scenarios =====
 * ===== */
/** Encapsulates a sequence of operations (enter/exit) as a test sc
enario. */
static final class Scenario {
    final String name;
    final List<Runnable> steps = new ArrayList<>();
    Scenario(String name) { this.name = name; }
    Scenario step(Runnable r) { steps.add(r); return this; }
    void run(StationLogger log) {
        log.log("\n=== Running Scenario: " + name + " (steps=" + st
eps.size() + ") ===");
        for (Runnable r : steps) r.run();
    }
}
/** Builds standard scenarios from your provided sequences. */
static final class ScenarioFactory {
    static Scenario VT(SubwayStation s, FlowController c, StationLo
gger log) {
        Scenario sc = new Scenario("VT");
        // Your baseline VT entries/exits (kept order; controller a
plied)
        int[] vtEnter = {

```

-200,

-270,

-135,

-474,

320,

-259,

-471,

172,

210,

402,

262,

-243,

-391,

-420,

-225,

55,

238,

-246,

261,

-121,

-387,

276,

110,

153,

-141,

-440,

47,

142,

-129,

-410,

373,

-171,

194,

358,

-492,

-489,

84,

204,

70,

-322,

334,

-476,

-183,

327,

```
-100,  
-215,  
-320,  
231,  
-102,  
-213,  
};  
int[] vtExit = {  
-491,  
75,  
-147,  
216,  
-343,  
422,  
262,  
161,  
-94,  
45,  
354,  
-473,  
181,  
-356,  
-75,  
463,  
-219,  
246,  
119,  
262,  
251,  
-168,  
450,  
276,  
-491,  
-144,  
-447,  
454,  
-43,  
197,  
-210,  
248,  
337,  
-207,  
-123,  
89,
```

```

        -326,
        -238,
        -16,
        482,
        299,
        -39,
        419,
        -257,
        -468,
        -226,
        -499,
        130,
        19,
        63,
    };

    final int[] time = { 8 * 60 }; // start 08:00
    for (int v : vtEnter) {
        sc.step(() -> {
            int req = c.preprocessEntryRequest(v, time[0]);
            s.enterStation(req);
            time[0] += 1; // 1 min step
        });
    }
    for (int v : vtExit) {
        sc.step(() -> {
            int req = c.preprocessExitRequest(v, time[0]);
            s.exitStation(req);
            time[0] += 1;
        });
    }
    return sc;
}

static Scenario UVT(SubwayStation s, FlowController c, StationL
ogger log) {
    // Use exactly the same sequence as provided for UVT in you
r example

    Scenario sc = new Scenario("UVT");
    int[] uvtEnter = {
        -200,
        -270,
        -135,
        -474,
        320,
        -259,
    };

```

-471,

172,

210,

402,

262,

-243,

-391,

-420,

-225,

55,

238,

-246,

261,

-121,

-387,

276,

110,

153,

-141,

-440,

47,

142,

-129,

-410,

373,

-171,

194,

358,

-492,

-489,

84,

204,

70,

-322,

334,

-476,

-183,

327,

-100,

-215,

-320,

231,

-102,

-213,

```
};  
int[] uvtExit = {  
    -491,  
    75,  
    -147,  
    216,  
    -343,  
    422,  
    262,  
    161,  
    -94,  
    45,  
    354,  
    -473,  
    181,  
    -356,  
    -75,  
    463,  
    -219,  
    246,  
    119,  
    262,  
    251,  
    -168,  
    450,  
    276,  
    -491,  
    -144,  
    -447,  
    454,  
    -43,  
    197,  
    -210,  
    248,  
    337,  
    -207,  
    -123,  
    89,  
    -326,  
    -238,  
    -16,  
    482,  
    299,  
    -39,
```

```

        419,
        -257,
        -468,
        -226,
        -499,
        130,
        19,
        63,
    };
    final int[] time = { 17 * 60 }; // 17:00
    for (int v : uvtEnter) {
        sc.step(() -> {
            int req = c.preprocessEntryRequest(v, time[0]);
            s.enterStation(req);
            time[0] += 2; // 2 min cadence
        });
    }
    for (int v : uvtExit) {
        sc.step(() -> {
            int req = c.preprocessExitRequest(v, time[0]);
            s.exitStation(req);
            time[0] += 2;
        });
    }
    return sc;
}

static Scenario Z3(SubwayStation s, FlowController c, StationLogger log) {
    // Use the same Z3 block from your baseline (enter/exit order preserved)
    Scenario sc = new Scenario("Z3");
    int[] z3Enter = {
        -200,
        -270,
        -135,
        -474,
        320,
        -259,
        -471,
        172,
        210,
        402,
        262,
        -243,
    };

```

```
-391,  
-420,  
-225,  
55,  
238,  
-246,  
261,  
-121,  
-387,  
276,  
110,  
153,  
-141,  
-440,  
47,  
142,  
-129,  
-410,  
373,  
-171,  
194,  
358,  
-492,  
-489,  
84,  
204,  
70,  
-322,  
334,  
-476,  
-183,  
327,  
-100,  
-215,  
-320,  
231,  
-102,  
-213,  
};  
int[] z3Exit = {  
    -491,  
    75,  
    -147,  
    216,
```

-343,

422,

262,

161,

-94,

45,

354,

-473,

181,

-356,

-75,

463,

-219,

246,

119,

262,

251,

-168,

450,

276,

-491,

-144,

-447,

454,

-43,

197,

-210,

248,

337,

-207,

-123,

89,

-326,

-238,

-16,

482,

299,

-39,

419,

-257,

-468,

-226,

-499,

130,

```

        19,
        63,
    };
    final int[] t = { 6 * 60 + 30 }; // 06:30
    for (int v : z3Enter) {
        sc.step(() -> {
            int req = c.preprocessEntryRequest(v, t[0]);
            s.enterStation(req);
            t[0] += 1;
        });
    }
    for (int v : z3Exit) {
        sc.step(() -> {
            int req = c.preprocessExitRequest(v, t[0]);
            s.exitStation(req);
            t[0] += 1;
        });
    }
    return sc;
}

static Scenario FT(SubwayStation s, FlowController c, StationLogger log, long seed) {
    // Fuzz Test: random large magnitudes, including negatives.
    Scenario sc = new Scenario("FT");
    Random r = new Random(seed);
    final int[] time = { 12 * 60 }; // noon
    for (int i = 0; i < 120; i++) {
        int val = r.nextBoolean()
            ? r.nextInt(800) - r.nextInt(1200) // may be negative
            : r.nextInt(2500); // big positive

        sc.step(() -> {
            int req = c.preprocessEntryRequest(val, time[0]);
            s.enterStation(req);
            time[0] += 1;
        });
        int out = r.nextInt(1200) - r.nextInt(900); // exit may be negative
        sc.step(() -> {
            int req = c.preprocessExitRequest(out, time[0]);
            s.exitStation(req);
            time[0] += 1;
        });
    }
}

```

```

        }
        return sc;
    }
}

/* =====
 * ===== Replay =====
 * ===== */
/** A simple deterministic RNG factory to reproduce runs. */
static final class DRand {
    static Random of(long seed) { return new Random(seed); }
    static ThreadLocalRandom threadLocal(long seed) {
        // not strictly deterministic across threads, but single-
        thread run is fine
        return ThreadLocalRandom.current();
    }
}

/* =====
 * ===== Runner/CLI =====
 * ===== */
/** Supported modes. */
enum Mode { VT, FT, Z3, UVT }
/** Runner ties together station, sensors, controller, and a given
scenario. */
static final class Runner {
    private final SubwayStation station;
    private final FlowController controller;
    private final StationLogger logger;
    private final CrowdSensor cx;
    Runner(SubwayStation station, FlowController controller, StationLogger logger, CrowdSensor cx) {
        this.station = station;
        this.controller = controller;
        this.logger = logger;
        this.cx = cx;
    }
    void runScenario(Scenario sc) {
        logger.log("Crowd sensor initial reading: " + String.format(
"%0.2f", cx.read()));
        sc.run(logger);
        logger.log("Crowd sensor final reading: " + String.format(
"%0.2f", cx.read()));
    }
    void printFinalSummary() {
        station.stats().print(logger);
    }
}

```

```

        logger.printSummary();
        // Optional dump of station embedded log:
        // station.printLog();
    }
}
/* =====
 * ===== Main =====
 * ===== */
public static void main(String[] args) {
    // Default configuration (you can override by CLI):
    Mode mode = Mode.UVT;
    long seed = 20251104L;    // deterministic seed (Tokyo time today)

    boolean consoleLog = true; // LOG=on/off
    // Parse simple CLI flags: MODE=VT|FT|Z3|UVT, SEED=<Long>, LOG=on/off
    for (String a : args) {
        String up = a.trim();
        if (up.startsWith("MODE=")) {
            String m = up.substring("MODE=".length()).trim().toUpperCase(Locale.ROOT);
            try { mode = Mode.valueOf(m); } catch (Exception ignore) {}
        } else if (up.startsWith("SEED=")) {
            try { seed = Long.parseLong(up.substring("SEED=".length()).trim()); } catch (Exception ignore) {}
        } else if (up.startsWith("LOG=")) {
            String v = up.substring("LOG=".length()).trim().toLowerCase(Locale.ROOT);
            consoleLog = v.equals("on") || v.equals("true") || v.equals("1");
        }
    }
    // Shared Logger
    StationLogger shared = new StationLogger();
    shared.setConsole(consoleLog);
    // Policy mirrors your baseline thresholds
    TicketGatePolicy policy = new TicketGatePolicy(
        0.80, // high density threshold
        0.30, // low density threshold
        3,    // high density => 3 open
        8,    // low density  => 8 open
        5     // normal       => 5 open
    );
}

```

```

        // Build station (your baseline: max=1000, initial=400)
        SubwayStation station = new SubwayStation(1000, 400, policy, shared);

        // Sensors & Controller
        CrowdSensor crowdSensor = new CrowdSensor(station::getCurrentPassengers, station::getMaxCapacity);
        FlowController controller = new FlowController(station, crowdSensor, shared);
        controller.setPeakHourBias(true);
        controller.setEmergencyCompression(true);
        // Runner
        Runner runner = new Runner(station, controller, shared, crowdSensor);

        // Choose scenario
        Scenario scenario;
        switch (mode) {
            case VT:
                scenario = ScenarioFactory.VT(station, controller, shared);
                break;
            case Z3:
                scenario = ScenarioFactory.Z3(station, controller, shared);
                break;
            case FT:
                scenario = ScenarioFactory.FT(station, controller, shared, seed);
                break;
            default:
                scenario = ScenarioFactory.UVT(station, controller, shared);
        }
        // scenario = ScenarioFactory.FT(station, controller, shared, seed);
    }
    // Execute
    shared.log("=== Subway Passenger Flow System (Mode=" + mode + ", Seed=" + seed + ") ===");
    runner.runScenario(scenario);
    runner.printFinalSummary();
    // ---
    - Optional: Quick health check lines to approach ~1000 Loc with clarity
    ----

        // Below are additional diagnostic probes and tiny utilities that are useful in

```

```

        // your SmartWell harnesses (kept minimal, readable, and self-
contained).
        final HealthChecks checks = new HealthChecks(shared);
        checks.checkPolicyConsistency(policy);
        checks.checkStationBounds(station);
        checks.checkCrowdSensor(crowdSensor);
        checks.printDone();
    }
    /* =====
    * ===== Health Checks =====
    * ===== */
    /**
    * Auxiliary consistency checks. These do not modify the model.
    * They are useful in research harnesses to quickly sanity-
verify a run.
    */
    static final class HealthChecks {
        private final StationLogger log;
        HealthChecks(StationLogger log) { this.log = log; }
        void checkPolicyConsistency(TicketGatePolicy p) {
            if (p.lowDensity < 0 || p.lowDensity > 1 || p.highDensity <
0 || p.highDensity > 1) {
                log.alert(AlertType.INFO, "Policy density thresholds ou
t of [0,1].");
            }
            if (p.lowDensity >= p.highDensity) {
                log.alert(AlertType.INFO, "Policy lowDensity >= highDen
sity; may produce unexpected switching.");
            }
            if (p.lowModeOpen <= 0 || p.highModeOpen <= 0 || p.normalOp
en <= 0) {
                log.alert(AlertType.INFO, "Policy open counts must be >
0.");
            }
        }
        void checkStationBounds(SubwayStation s) {
            if (s.getMaxCapacity() <= 0) {
                log.alert(AlertType.INFO, "Station capacity invalid: "
+ s.getMaxCapacity());
            }
            if (s.getCurrentPassengers() < 0) {
                log.alert(AlertType.INFO, "Station current < 0: " + s.g
etCurrentPassengers());
            }
        }
    }

```

```

    }
    void checkCrowdSensor(CrowdSensor c) {
        double r = c.read();
        if (Double.isNaN(r) || r < 0 || r > 1.0) {
            log.alert(AlertType.SENSOR_FAULT, "CrowdSensor reading
abnormal: " + r);
        } else {
            log.log("HealthCheck: CrowdSensor OK: " + String.format
("%.2f", r));
        }
    }
    void printDone() {
        log.log("\n[HealthChecks] Completed.");
    }
}
}

```

- Coverage report

1. Test case number: 100
2. V-Method testing branch coverage: 62.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c123_SubwayPassengerFlowSystemEx.java	49.2 %	59	61	120
▼ c123_SubwayPassengerFlowSystemEx	11.1 %	2	16	18
main(String[])	11.1 %	2	16	18
> ScenarioFactory	25.0 %	4	12	16
> HealthChecks	50.0 %	13	13	26
▼ FlowController	62.5 %	20	12	32
▲ preprocessEntryRequest(int, int)	58.3 %	7	5	12
■ isPeakHour(int)	62.5 %	5	3	8
▲ preprocessExitRequest(int, int)	66.7 %	8	4	12
▲ FlowController(SubwayStation, Cr		0	0	0
▲ setEmergencyCompression(boole		0	0	0
▲ setPeakHourBias(boolean)		0	0	0
> SubwayStation	70.0 %	7	3	10
> StationLogger	75.0 %	6	2	8
> TicketGatePolicy	83.3 %	5	1	6

3. Vibration testing instruction coverage: 85.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c123_SubwayPassengerFlowSystemEx.java	53.1 %	1,557	1,377	2,934
▼ c123_SubwayPassengerFlowSystemEx	55.9 %	114	90	204
> ScenarioFactory	31.2 %	504	1,112	1,616
main(String[])	56.7 %	114	87	201
> HealthChecks	69.7 %	83	36	119
> SubwayStation	83.5 %	238	47	285
▼ FlowController	85.3 %	116	20	136
▲ preprocessEntryRequest(int, int)	77.8 %	35	10	45
▲ preprocessExitRequest(int, int)	77.8 %	35	10	45
▲ FlowController(SubwayStation, Cr	100.0 %	18	0	18
■ isPeakHour(int)	100.0 %	20	0	20
▲ setEmergencyCompression(boole	100.0 %	4	0	4
▲ setPeakHourBias(boolean)	100.0 %	4	0	4
> TicketGatePolicy	93.8 %	45	3	48
> StationLogger	94.2 %	131	8	139

4. Fuzz testing branch coverage: 37.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c123_SubwayPassengerFlowSystemEx	11.1 %	2	16	18
main(String[])	11.1 %	2	16	18
> ScenarioFactory	25.0 %	4	12	16
▼ FlowController	37.5 %	12	20	32
▲ preprocessExitRequest(int, int)	33.3 %	4	8	12
■ isPeakHour(int)	37.5 %	3	5	8
▲ preprocessEntryRequest(int, int)	41.7 %	5	7	12
▲ FlowController(SubwayStation, Cr		0	0	0
▲ setEmergencyCompression(boole		0	0	0
▲ setPeakHourBias(boolean)		0	0	0
> HealthChecks	50.0 %	13	13	26
> StationLogger	75.0 %	6	2	8
> SubwayStation	80.0 %	8	2	10
> TicketGatePolicy	83.3 %	5	1	6
> Scenario	100.0 %	2	0	2

5. Fuzz testing instruction coverage: 59.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c123_SubwayPassengerFlowSystemEx	56.1 %	115	90	205
> ScenarioFactory	6.2 %	101	1,515	1,616
main(String[])	56.9 %	115	87	202
▼ FlowController	59.6 %	81	55	136
▲ preprocessExitRequest(int, int)	33.3 %	15	30	45
▲ preprocessEntryRequest(int, int)	55.6 %	25	20	45
■ isPeakHour(int)	75.0 %	15	5	20
▲ FlowController(SubwayStation, Cr	100.0 %	18	0	18
▲ setEmergencyCompression(boole	100.0 %	4	0	4
▲ setPeakHourBias(boolean)	100.0 %	4	0	4
> HealthChecks	69.7 %	83	36	119
> SubwayStation	91.6 %	261	24	285
> TicketGatePolicy	93.8 %	45	3	48
> StationLogger	94.2 %	131	8	139
> Alert	100.0 %	21	0	21

6. Z3 branch coverage: 37.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c123_SubwayPassengerFlowSystemEx	11.1 %	2	16	18
main(String[])	11.1 %	2	16	18
> ScenarioFactory	25.0 %	4	12	16
▼ FlowController	37.5 %	12	20	32
▲ preprocessExitRequest(int, int)	33.3 %	4	8	12
■ isPeakHour(int)	37.5 %	3	5	8
▲ preprocessEntryRequest(int, int)	41.7 %	5	7	12
▲ FlowController(SubwayStation, Cr		0	0	0
▲ setEmergencyCompression(boole		0	0	0
▲ setPeakHourBias(boolean)		0	0	0
> HealthChecks	50.0 %	13	13	26
> StationLogger	75.0 %	6	2	8
> SubwayStation	80.0 %	8	2	10
> TicketGatePolicy	83.3 %	5	1	6
> Scenario	100.0 %	2	0	2

7. Z3 instruction coverage: 59.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c123_SubwayPassengerFlowSystemEx.java	38.9 %	1,143	1,792	2,935
▼ c123_SubwayPassengerFlowSystemEx	56.1 %	115	90	205
> ScenarioFactory	6.2 %	101	1,515	1,616
main(String[])	56.9 %	115	87	202
▼ FlowController	59.6 %	81	55	136
▲ preprocessExitRequest(int, int)	33.3 %	15	30	45
▲ preprocessEntryRequest(int, int)	55.6 %	25	20	45
■ isPeakHour(int)	75.0 %	15	5	20
▲ FlowController(SubwayStation, Cr	100.0 %	18	0	18
▲ setEmergencyCompression(boole	100.0 %	4	0	4
▲ setPeakHourBias(boolean)	100.0 %	4	0	4
> HealthChecks	69.7 %	83	36	119
> SubwayStation	91.6 %	261	24	285
> TicketGatePolicy	93.8 %	45	3	48
> StationLogger	94.2 %	131	8	139

8. Unimproved V-method branch coverage: 62.5%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c123_SubwayPassengerFlowSystemEx.java		49.2 %	59	61	120
▼ c123_SubwayPassengerFlowSystemEx		11.1 %	2	16	18
main(String[])		11.1 %	2	16	18
> ScenarioFactory		25.0 %	4	12	16
> HealthChecks		50.0 %	13	13	26
▼ FlowController		62.5 %	20	12	32
▲ preprocessEntryRequest(int, int)		58.3 %	7	5	12
■ isPeakHour(int)		62.5 %	5	3	8
▲ preprocessExitRequest(int, int)		66.7 %	8	4	12
▲ FlowController(SubwayStation, Cr			0	0	0
▲ setEmergencyCompression(boole			0	0	0
▲ setPeakHourBias(boolean)			0	0	0
> SubwayStation		70.0 %	7	3	10
> StationLogger		75.0 %	6	2	8
> TicketGatePolicy		83.3 %	5	1	6

9. Unimproved V-method instruction coverage: 85.3%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c123_SubwayPassengerFlowSystemEx.java		53.1 %	1,557	1,377	2,934
▼ c123_SubwayPassengerFlowSystemEx		55.9 %	114	90	204
> ScenarioFactory		31.2 %	504	1,112	1,616
main(String[])		56.7 %	114	87	201
> HealthChecks		69.7 %	83	36	119
> SubwayStation		83.5 %	238	47	285
▼ FlowController		85.3 %	116	20	136
▲ preprocessEntryRequest(int, int)		77.8 %	35	10	45
▲ preprocessExitRequest(int, int)		77.8 %	35	10	45
▲ FlowController(SubwayStation, Cr		100.0 %	18	0	18
■ isPeakHour(int)		100.0 %	20	0	20
▲ setEmergencyCompression(boole		100.0 %	4	0	4
▲ setPeakHourBias(boolean)		100.0 %	4	0	4
> TicketGatePolicy		93.8 %	45	3	48
> StationLogger		94.2 %	131	8	139